Московский государственный технический университет
им. Н. Э. Баумана

Проект СЭЛФ

МОДЕЛЬ ИНТЕРФЕРОМЕТРА ФАБРИ — ПЕРО

ЧАСТЬ 1. РАСЧЁТНАЯ ЧАСТЬ ПРОГРАММЫ

Методическое пособие по коду программы

Разобрана расчётная часть программы: как формулы физики записаны на Python и numpy, как программа сама измеряет характеристики прибора по графикам, какие приёмы позволяют считать картину точно и быстро даже на старом компьютере, как проверяется правильность расчётов. Каждый раздел — формула, настоящий код из программы, разбор по строкам и краткая формулировка для защиты.

Визуальная часть (окно, графики, анимация, сборка) разобрана во второй части пособия.

Москва, 2026

КАК УСТРОЕНА ПРОГРАММА

Программа написана на Python. Для расчётов используется библиотека **numpy** (быстрые операции над массивами чисел), для окна и рисования — **Qt** (PyQt6, а на Windows 7 — PyQt5). Код разделён на модули — файлы, у каждого своя задача (рис. 1). Расчётная часть — это модули `physics.py` и `fmt.py`: в них нет ни одной строчки про окно, поэтому их можно проверить и понять отдельно.

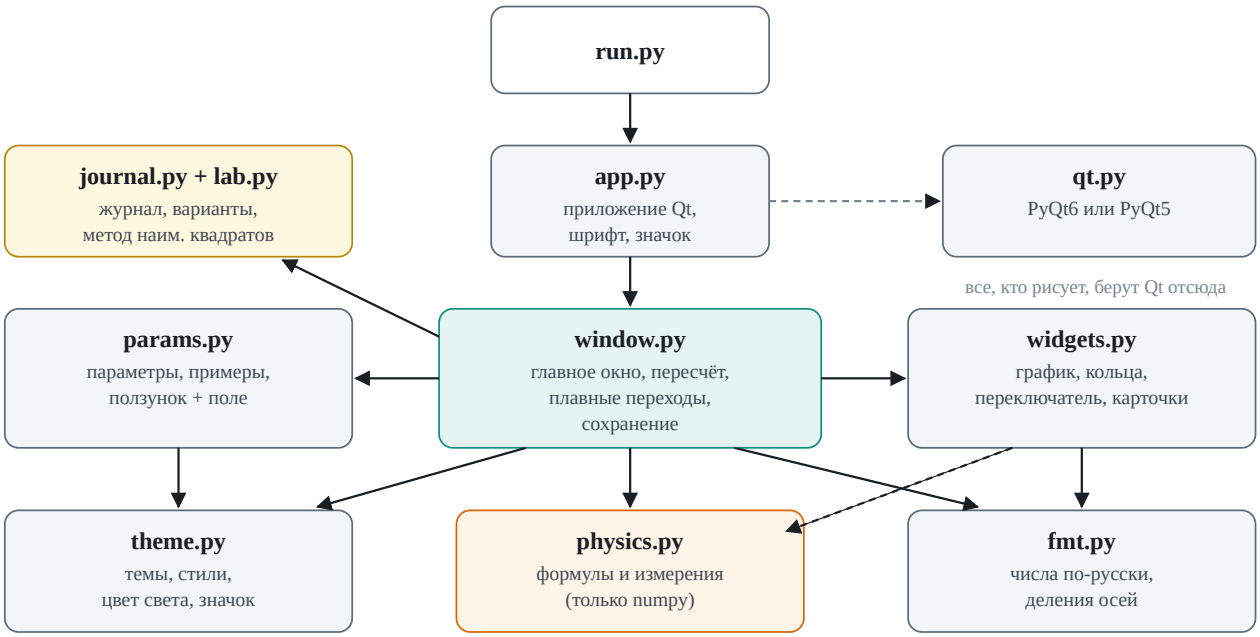


Рис. 1. Модули программы. Стрелка «A → B» означает «A пользуется B»

Таблица 1

Модули программы

Файл	Что в нём	Строк
`physics.py`	формулы интерферометра и измерения по графикам	≈ 230
`fmt.py`	запись чисел по-русски, «круглые» деления осей	≈ 90
`theme.py`	тёмная и светлая темы, цвет света по длине волны, значок	≈ 280
`widgets.py`	график, картина колец, переключатель, карточки	≈ 600
`params.py`	список параметров, готовые примеры, строка «ползунок + поле»	≈ 160
`window.py`	главное окно, пересчёт, плавные переходы, сохранение	≈ 750
`lab.py`	варианты лабораторной, метод наименьших квадратов, обработка серий	≈ 170
`journal.py`	журнал измерений (окно)	≈ 350
`app.py, qt.py`	запуск; выбор PyQt6 или PyQt5	≈ 80

Единицы. Внутри расчёта все величины — в СИ: метры, радианы, герцы. Пользователь вводит нанометры и миллиметры; перевод делается один раз в начале пересчёта (`window.recalc`): например, `lam = v["lam"] * 1e-9`. Так в формулах нет «магических» множителей, а ошибки с единицами почти исключены.

Азбука numpy за одну страницу

Всё быстрое в программе держится на одной идее numpy: **действие сразу над целым массивом**. Цикл Python обрабатывает около десяти миллионов чисел в секунду, а numpy — сотни миллионов, потому что внутренний цикл написан на языке C.

```
x = np.linspace(0, 1, 5)          # массив [0, 0.25, 0.5, 0.75, 1]
y = np.sin(x) ** 2                # синус и квадрат — сразу для всех пяти чисел
mask = y > 0.2                    # массив True/False: где условие выполнено
idx = np.flatnonzero(mask)        # номера элементов, где True
col = x[:, None]; row = x[None, :]  # столбец 5×1 и строка 1×5
table = col * row                 # «растяжение» (broadcasting): таблица 5×5
```

Последние две строки — приём *broadcasting*: numpy сам «растягивает» столбец и строку до таблицы. Им программа считает сразу все пиксели картинки: строка — номера столбцов, столбец — номера строк.

1. ФИЗИЧЕСКАЯ МОДЕЛЬ

Модуль `physics.py` повторяет формулы теоретической части лабораторной работы. Соответствие формул и функций — в табл. 2.

Таблица 2

Формулы и функции

Формула	Функция
$F = 4R \,/\, (1-R)^2$	`coefficient_f(R)`
$T_{\max} = (1-R-A)^2 \,/\, (1-R)^2$	`peak_transmission(R, A)`
$\delta = 4\pi d\sqrt{n^2-\sin^2\theta}\ / \lambda$	`phase(lam, d, n, sin_out)`
$T = T_{\max} \,/\, (1 + F\sin^2(\delta/2))$	`airy(delta, F, tmax)`
$m = 2d\sqrt{n^2-\sin^2\theta}\ / \lambda$	`order_at(r, lam, d, n, f)`
$\Delta\lambda,\ w,\ F,\ A,\ m_0, \ldots$	`theory(lam, d, n, R, f, A)`
$2d\sqrt{n^2-\sin^2\theta} = m\lambda,\ r = f\,\mathrm{tg}\,\theta$	`ring_radii(…)`

1.1. Коэффициент резкости, высота пиков, разность фаз

fabry_perot/physics.py — coefficient_f

```
def coefficient_f(R):
    return 4.0 * R / (1.0 - R) ** 2
```

fabry_perot/physics.py — peak_transmission

```
def peak_transmission(R, A=0.0):
    return (max(1.0 - R - A, 0.0) / (1.0 - R)) ** 2
```

В `peak_transmission` стоит `max(1.0 - R - A, 0.0)`: зеркало не может отразить и поглотить больше 100 % света. Если пользователь задаст $R + A > 1$, пропускание будет нулевым, а не отрицательным.

fabry_perot/physics.py — phase

```python
def phase(lam, d, n, sin_out=0.0):
    return 4.0 * np.pi * d * np.sqrt(n * n - sin_out * sin_out) / lam
```

Функция принимает и одно число, и целый массив: если `lam` — массив из тысячи длин волн, вернётся массив из тысячи фаз. Благодаря этому одна и та же формула служит и для графика $T(\lambda)$ (массив длин волн), и для колец (массив углов $\sin\theta$).

1.2. Функция Эйри

fabry_perot/physics.py — airy

```python
def airy(delta, F, tmax=1.0):
    return tmax / (1.0 + F * np.sin(delta / 2.0) ** 2)
```

> **КАК СКАЗАТЬ НА ЗАЩИТЕ**
> «Пропускание считается по формуле Эйри. Она получается сложением бесконечного числа лучей, отражённых между зеркалами: их амплитуды — геометрическая прогрессия со знаменателем $Re^{i\delta}$. В программе формула записана одной строкой и работает сразу для массива точек.»

1.3. Характеристики прибора по формулам

fabry_perot/physics.py — theory

```python
def theory(lam, d, n, R, f, A=0.0):
    F = coefficient_f(R)
    tmax = peak_transmission(R, A)
    fsr = lam ** 2 / (2 * n * d)
    res = {
        "F": F,
        "tmax": tmax,
        "m0": 2 * n * d / lam,
        "T": float(airy(phase(lam, d, n), F, tmax)),
        "fsr": fsr,
        "fsr_nu": C / (2 * n * d),
        "finesse_approx": math.pi * math.sqrt(R) / (1 - R),
        "contrast": 1 + F,
        "dr2": f * f * n * lam / d,
        "width": None, "finesse": None, "resolving": None,
    }
    if F > 1:
        half = 4 * math.asin(1 / math.sqrt(F))
        res["finesse"] = 2 * math.pi / half
        res["width"] = fsr / res["finesse"]
        res["resolving"] = lam / res["width"]
    return res
```

Функция возвращает словарь — набор пар «имя: значение». Ширина пика существует, только если провалы между пиками опускаются ниже половины высоты: $T_{\min} = T_{\max} / (1 + F) < T_{\max} / 2$, т. е. $F > 1$ $(R > 0,172)$. При более плохих зеркалах программа честно пишет прочерк, а не выдумывает число.

1.4. Радиусы колец по точной формуле

fabry_perot/physics.py — ring_radii

```python
def ring_radii(lam, d, n, f, r_max, limit=500):
    m0 = 2 * n * d / lam
    a = lam / (2 * d)
    first = math.ceil(m0) - 1
    m = np.arange(first, max(first - limit, 0), -1, dtype=float)
```

```
    s2 = a * (m0 - m) * (n + m * a)
    s2 = s2[(s2 > 0) & (s2 < 1)]
    r = f * np.sqrt(s2 / (1 - s2))
    return r[r <= r_max]
```

Из условия максимума $2d\sqrt{n^2 - \sin^2\theta} = m\lambda$ находим $\sin^2\theta = n^2 - (ma)^2$, где $a = \lambda\,/\,(2d)$. Прямое вычитание двух почти равных чисел (n^2 и $(ma)^2$ отличаются в шестом-седьмом знаке) теряет точность. Поэтому разность записана как произведение: $n^2 - (ma)^2 = (n - ma)(n + ma) = a\,(m_0 - m)(n + ma)$, так как $n = am_0$. Множитель $m_0 - m$ — небольшое число, и точность сохраняется полностью.

> **КАК СКАЗАТЬ НА ЗАЩИТЕ**
>
> «Радиусы колец по формуле: из условия максимума находим синус угла для каждого порядка, радиус — $r = f\,\mathrm{tg}\,\theta$. Разность квадратов записана как произведение, чтобы не терять точность при вычитании близких чисел.»

2. ИЗМЕРЕНИЯ «ПО ГРАФИКУ»

Главная особенность программы: она не только считает характеристики по формулам, но и **измеряет** их по построенным кривым, как экспериментатор по осциллограмме. Совпадение измерения и формулы — проверка правильности всей модели.

2.1. Поиск максимумов

fabry_perot/physics.py — find_peaks

```
def find_peaks(x, y):
    i = np.flatnonzero((y[1:-1] > y[:-2]) & (y[1:-1] >= y[2:])) + 1
    return i, x[i], y[i]
```

Максимум — точка, которая выше левой соседки и не ниже правой: `(y[1:-1] > y[:-2]) & (y[1:-1] >= y[2:])`. Здесь `y[1:-1]` — все точки, кроме крайних, `y[:-2]` — их левые соседки, `y[2:]` — правые. Сравнение сразу всех точек — одна операция numpy вместо цикла.

За вершину пика берётся сама найденная точка. Настоящая вершина может лежать между точками сетки, но точки стоят очень густо — не меньше 30 на ширину пика (`curve_samples`, раздел 2.2), поэтому ошибка меньше 1/60 ширины пика и на результат не влияет.

2.2. Расстояние между пиками, ширина, контраст

fabry_perot/physics.py — measure_curve

```
def measure_curve(x, T, x_line):
    res = {"fsr": None, "width": None, "contrast": None, "peaks": None, "half": None}
    if T.size < 3 or T.max() <= 0:
        return res
    idx, pos, height = find_peaks(x, T)
    if len(pos) < 2:
        return res
    k = int(np.clip(np.searchsorted(pos, x_line), 1, len(pos) - 1))
    res["fsr"] = pos[k] - pos[k - 1]
    res["peaks"] = (pos[k - 1], pos[k])
    j = k if abs(pos[k] - x_line) < abs(pos[k - 1] - x_line) else k - 1
    top, i = height[j], idx[j]
```

```
    res["contrast"] = top / T.min()
    half = top / 2
    left = np.flatnonzero(T[:i] < half)
    right = np.flatnonzero(T[i:] < half)
    if left.size and right.size:
        a, b = left[-1], i + right[0]
        x_left = np.interp(half, [T[a], T[a + 1]], [x[a], x[a + 1]])
        x_right = np.interp(half, [T[b], T[b - 1]], [x[b], x[b - 1]])
        res["width"] = x_right - x_left
        res["half"] = (x_left, x_right, half)
    return res
```

Порядок действий: найти все максимумы; выбрать два соседних вокруг λ_1 (`np.searchsorted` находит место λ_1 в упорядоченном списке пиков); их разность — $\Delta\lambda$. Затем у ближайшего к λ_1 пика ищется уровень половины высоты: последняя точка ниже половины слева и первая справа. Точное место пересечения находится линейной интерполяцией между соседними точками (`np.interp`). Разность — ширина w.

Сколько точек брать? Функция `curve_samples` требует не меньше 30 точек на ширину пика — тогда ошибка интерполяции меньше сотой доли процента. Измерение идёт только в окрестности $\pm 1,5\,\Delta\lambda$ вокруг λ_1, поэтому даже при $R = 0,99$ точек около 28 тысяч.

fabry_perot/physics.py — curve_samples

```
def curve_samples(span, width, fsr):
    return int(min(max(30 * span / (width or fsr), 20000), 2e6))
```

2.3. Радиусы колец

Радиусы колец программа берёт из точной формулы `ring_radii` (раздел 1.4): кольца на картинке рисуются той же функцией Эйри, поэтому их положения с формулой совпадают. Независимая проверка «по графику» остаётся для кривой пропускания: $\Delta\lambda$, w и контраст измеряются по точкам (раздел 2.2).

3. ТОЧНЫЕ ФОРМУЛЫ ДЛЯ БЫСТРОЙ КАРТИНКИ

Картинку на экране можно было бы строить «в лоб»: посчитать пропускание в миллионе точек и нарисовать. Так делала первая версия программы; в тяжёлых режимах (тысячи колец, $R = 0,99$) это занимало до 350 мс на кадр. Вторая версия использует два математических приёма и тратит около 10 мс при любых параметрах.

3.1. Наибольшее и наименьшее значение в столбце пикселей

На экране кривая рисуется столбцами пикселей. Для каждого столбца нужно знать только две вещи: где кривая в этом столбце выше всего и где ниже всего (рис. 3). Для функции Эйри это известно без перебора точек: если внутри отрезка фаз $[\delta_a, \delta_b]$ есть $\delta = 2\pi m$, наибольшее значение равно $T_{\max}$; если есть $\delta = \pi(2m+1)$, наименьшее равно $T_{\max} \,/\, (1+F)$; иначе крайние значения — на концах отрезка, потому что между пиком и провалом функция монотонна.

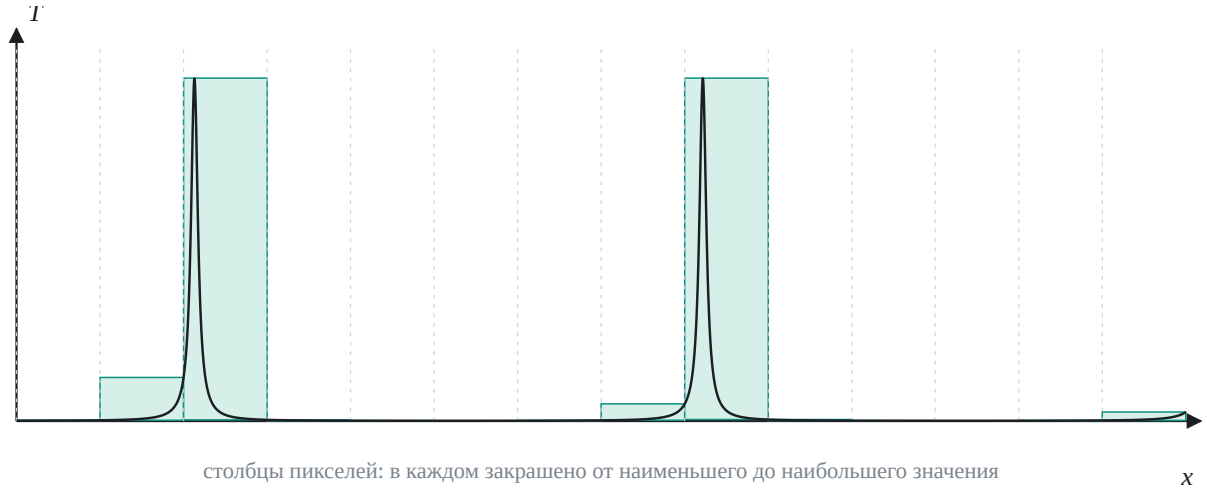


Рис. 3. Каждый столбец пикселей закрашивается от наименьшего до наибольшего значения кривой в нём — узкий пик не пропадёт, даже если он тоньше пикселя

fabry_perot/physics.py — airy_range

```python
def airy_range(edges, F, tmax=1.0):
    a, b = np.minimum(edges[:-1], edges[1:]), np.maximum(edges[:-1], edges[1:])
    ta, tb = airy(a, F, tmax), airy(b, F, tmax)
    period = 2 * np.pi
    has_peak = np.floor(b / period) * period >= a
    has_dip = np.floor((b - np.pi) / period) * period + np.pi >= a
    hi = np.where(has_peak, tmax, np.maximum(ta, tb))
    lo = np.where(has_dip, tmax / (1 + F), np.minimum(ta, tb))
    return lo, hi
```

Проверка «есть ли $2\pi m$ на отрезке $[a, b]$»: берём ближайшее снизу к b кратное 2π — `np.floor(b / period) * period` — и смотрим, не меньше ли оно a. Вся функция — десяток действий numpy над массивом из тысячи столбцов: доли миллисекунды.

3.2. Средняя яркость пикселя через первообразную

Пиксель картины колец покрывает интервал радиусов, внутри которого у края экрана помещаются десятки тонких колец. Если взять яркость в одной точке пикселя, получится муар — ложные узоры. Правильно брать **среднее** по пикселю:

$$\bar{T} = \frac{1}{\delta_2 - \delta_1} \int_{\delta_1}^{\delta_2} \frac{T_{\max}\, d\delta}{1 + F \sin^2(\delta \,/\, 2)}. \tag{2}$$

У функции Эйри есть первообразная. Подстановка $t = \operatorname{tg}(\delta \,/\, 2)$ даёт на одном периоде $(-\pi, \pi)$

$$\int \frac{d\delta}{1 + F \sin^2(\delta \,/\, 2)} = \frac{2}{\sqrt{1+F}} \operatorname{arctg}\left(\sqrt{1+F} \operatorname{tg}\frac{\delta}{2} \right). \tag{3}$$

Арктангенс «сбрасывается» на каждом периоде, поэтому непрерывная первообразная $G(\delta)$ собирается так: $\delta = 2\pi k + \varphi$, $\varphi \in [-\pi, \pi)$, и за каждый полный период прибавляется $2\pi \,/\, \sqrt{1+F}$ (рис. 4):

$$G(\delta) = \frac{2}{\sqrt{1+F}} \left(\pi k + \operatorname{arctg}\left(\sqrt{1+F} \operatorname{tg}\frac{\varphi}{2} \right) \right), \qquad \bar{T} = T_{\max} \frac{G(\delta_2) - G(\delta_1)}{\delta_2 - \delta_1}. \tag{4}$$

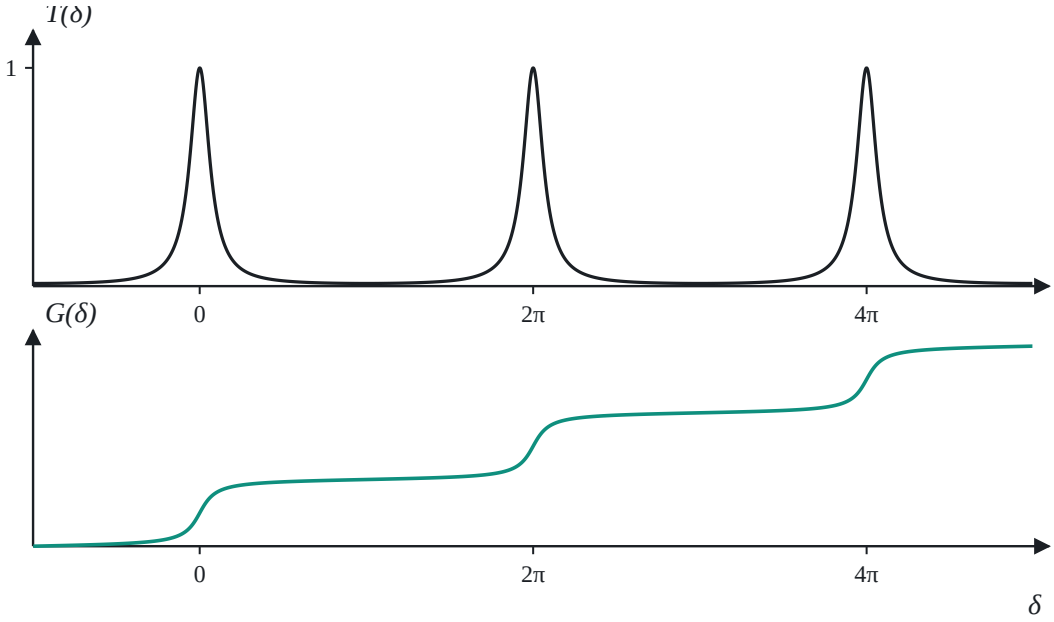


Рис. 4. Функция Эйри (вверху) и её непрерывная первообразная $G(\delta)$ (внизу): ступенька высотой $2\pi \,/\, \sqrt{1+F}$ на каждый пик

fabry_perot/physics.py — airy_mean

```python
def airy_mean(d1, d2, F, tmax=1.0):
    q = np.sqrt(1.0 + F)
    k1 = np.floor(d1 / (2 * np.pi) + 0.5)
    k2 = np.floor(d2 / (2 * np.pi) + 0.5)
    a1 = np.arctan(q * np.tan((d1 - 2 * np.pi * k1) / 2))
    a2 = np.arctan(q * np.tan((d2 - 2 * np.pi * k2) / 2))
    width = d2 - d1
    tiny = np.abs(width) < 1e-9
    mean = (2 / q) * (np.pi * (k2 - k1) + (a2 - a1)) / np.where(tiny, 1.0, width)
    return tmax * np.where(tiny, airy((d1 + d2) / 2, F), mean)
```

Тонкие места. Фазы δ бывают порядка 10^6 радиан, а нужна разность двух близких значений G. Поэтому целая часть $\pi(k_2 - k_1)$ и дробная $a_2 - a_1$ вычитаются отдельно — так не теряется точность. Если пиксель попал в центр картины, где фаза почти не меняется ($|\delta_2 - \delta_1| < 10^{-9}$), вместо деления на почти ноль берётся значение в середине.

Проверка: за целый период среднее равно $1 \,/\, \sqrt{1+F}$ — это известное среднее пропускание интерферометра.

> **КАК СКАЗАТЬ НА ЗАЩИТЕ**
>
> «Яркость пикселя — среднее по пикселю, иначе на тонких кольцах появляется муар. Среднее считается точно, через первообразную функции Эйри: одна формула на пиксель вместо сотен точек. Поэтому картинка строится за миллисекунды при любом числе колец.»

3.3. Анимация зеркала без скачков

fabry_perot/physics.py — seamless_shift

```python
def seamless_shift(lam, n, limit):
    step = lam / (2 * n)
    return max(step, math.floor(limit / step) * step)
```

Анимация двигает зеркало от 0 до 1000 нм и начинает сначала. Если просто вернуться в 0, картинка скачком изменится. Но через каждые $\lambda / (2n)$ картина повторяется (порядок в центре меняется ровно на 1). Поэтому цикл обрезается на наибольшем кратном $\lambda / (2n)$ — и переход «конец → начало» незаметен.

3.4. Экран не в фокусе (режим лабораторной)

В лабораторной линза выбирается из набора, а расстояние от линзы до экрана L ставит студент. Пучок шириной $D = 10$ мм линза собирает в точку на расстоянии f; на экране при $L \neq f$ получается кружок диаметром $D \mid L - f \mid \ / f$. Луч через центр линзы попадает в точку $r = L \operatorname{tg} \theta$. Яркость точки экрана — среднее картины по этому кружку; программа усредняет по отрезку радиусов $[r - \rho,\ r + \rho]$, и среднее снова даёт первообразная, поэтому расфокусировка не замедляет программу.

fabry_perot/physics.py — defocus_radius

```python
def defocus_radius(L, f, aperture=APERTURE):
    return aperture / 2 * abs(L - f) / f
```

fabry_perot/physics.py — screen_mean

```python
def screen_mean(r1, r2, lam, d, n, L, F, tmax=1.0):
    s1, s2 = r1 / np.hypot(r1, L), r2 / np.hypot(r2, L)
    return airy_mean(phase(lam, d, n, s1), phase(lam, d, n, s2), F, tmax)
```

3.5. Критерий разрешения по провалу

fabry_perot/physics.py — dip_ratio

```python
def dip_ratio(dlam, fsr, F):
    shift = 2 * np.pi * dlam / fsr
    x = np.linspace(-shift, 2 * shift, 6001)
    total = airy(x, F) + airy(x - shift, F)
    middle = float(airy(shift / 2, F) * 2)
    return middle / float(total.max())
```

Две линии одинаковой яркости дают сумму двух пиков; отношение яркости в середине между ними к максимуму — глубина провала. Предел разрешения — такое $\delta\lambda$, при котором провал равен 0,81. Для функции Эйри это $\delta\lambda_{\min} \approx 1,03\, w$, и `rayleigh_limit` считает его прямо по этой формуле. Проверка: при таком $\delta\lambda$ функция `dip_ratio` даёт 0,810 ± 0,001 для $R = 0,8 \ldots 0,99$.

fabry_perot/physics.py — rayleigh_limit

```python
def rayleigh_limit(fsr, F):
    if F <= 1:
        return None
    width = fsr * 4 * math.asin(1 / math.sqrt(F)) / (2 * np.pi)
    return 1.03 * width
```

4. ЛАБОРАТОРНАЯ РАБОТА: ВАРИАНТЫ И МЕТОД НАИМЕНЬШИХ КВАДРАТОВ

4.1. Варианты со скрытыми величинами

fabry_perot/lab.py — Variant.__init__

```
def __init__(self, code):
    self.code = int(code)
    rng = random.Random(f"fabry-perot-variant-{self.code}")
    self.lam = round(rng.uniform(450.0, 690.0), 1)
    self.dlam = float(round(rng.uniform(300.0, 900.0)))
    self.A = rng.choice([0.002, 0.003, 0.004, 0.005, 0.006, 0.008, 0.010])
    rng.choice([100, 120, 150, 180, 200])
    rng.choice([0.02, 0.03, 0.05, 0.08])
    options = [(d, f) for d in (2.0, 3.0, 4.0, 5.0, 6.0) for f in (150.0, 200.0, 250.0, 300.0)]
    rng.shuffle(options)
    for d, f in options:
        if len(ring_radii(self.lam * 1e-9, d * 1e-3, 1.0, f * 1e-3, 0.010)) >= 7:
            break
    self.d_rings, self.f_rings = d, f
```

Скрытые значения получаются из номера варианта генератором случайных чисел, «засеянным» строкой `"fabry-perot-variant-7"`. Такой генератор на любом компьютере и в любой версии Python выдаёт одну и ту же последовательность — поэтому вариант 7 у студента и в таблице ответов преподавателя один и тот же, а хранить сами значения нигде не нужно. Параметры задания с кольцами подбираются так, чтобы на экране было не меньше семи колец.

4.2. Прямая по методу наименьших квадратов

fabry_perot/lab.py — fit_line

```
def fit_line(xs, ys, through_origin=False):
    n = len(xs)
    if n < (1 if through_origin else 2):
        return None
    if through_origin:
        sxx = sum(x * x for x in xs)
        if sxx == 0:
            return None
        b = sum(x * y for x, y in zip(xs, ys)) / sxx
        a = 0.0
        dof = n - 1
        s2 = sum((y - b * x) ** 2 for x, y in zip(xs, ys)) / dof if dof > 0 else float("nan")
        sb, sa = math.sqrt(s2 / sxx) if dof > 0 else float("nan"), 0.0
    else:
        mx, my = sum(xs) / n, sum(ys) / n
        sxx = sum((x - mx) ** 2 for x in xs)
        if sxx == 0:
            return None
        b = sum((x - mx) * (y - my) for x, y in zip(xs, ys)) / sxx
        a = my - b * mx
        dof = n - 2
        s2 = sum((y - a - b * x) ** 2 for x, y in zip(xs, ys)) / dof if dof > 0 else float("nan")
        sb = math.sqrt(s2 / sxx) if dof > 0 else float("nan")
        sa = math.sqrt(s2 * (1 / n + mx * mx / sxx)) if dof > 0 else float("nan")
    t = student(dof)
    return {"a": a, "b": b, "sa": sa, "sb": sb, "n": n, "dof": dof, "t": t, "da": t * sa, "db": t
* sb}
```

Функция реализует формулы (19), (20) лабораторной работы: наклон, свободный член, их стандартные отклонения и доверительные границы с коэффициентом Стьюдента. Для прямой через начало координат число степеней свободы на единицу больше. Проверка в тестах — сравнение с `numpy.polyfit`.

4.3. Искомая величина по наклону

fabry_perot/lab.py — derived

```
def derived(key, fit, params, known):
    if fit is None or not fit["b"] or not math.isfinite(fit["b"]):
        return None
    b, db = fit["b"], fit["db"]
    n = params["n"]
    if key == "fsr":
        lam = math.sqrt(2 * n * b * 1e-15) * 1e9
        return ("λ", lam, lam * db / (2 * b), "нм")
    if key == "width":
        return ("Δλ", math.pi * b, math.pi * db, "пм")
    if key == "rings":
        lam = b * params["d"] / (params["f"] ** 2 * n) * 1e6
        return ("λ", lam, lam * db / b, "нм")
    if key == "shift":
        return ("λ", 2 * n * b, 2 * n * db, "нм")
    return None
```

Каждая ветка — формула из обработки результатов лабораторной, с переводом единиц: например, наклон $\Delta\lambda(1\ /\ d)$ в пм·мм равен 10^{-15} м². Погрешность пересчитывается по правилу для степени: у $\lambda = \sqrt{2nb}$ относительная погрешность вдвое меньше, чем у b.

5. ЧИСЛА ПО-РУССКИ И ДЕЛЕНИЯ ОСЕЙ

fabry_perot/fmt.py — num

```
def num(x, digits=4):
    if x is None:
        return "—"
    if isinstance(x, (int, np.integer)):
        return f"{x:,}".replace(",", " ")
    if not math.isfinite(x):
        return "—"
    if x == 0:
        return "0"
    power = math.floor(math.log10(abs(x)))
    if -3 <= power < 6:
        text = f"{x:,.{max(digits - 1 - power, 0)}f}".replace(",", " ")
    else:
        text = f"{x / 10 ** power:.{digits - 1}f}·10" + str(power).translate(SUP)
    return text.replace(".", ",").replace("-", "−")
```

Число записывается с четырьмя значащими цифрами, десятичной запятой, пробелом между тысячами и настоящим знаком минус «−». Очень большие и малые числа — через степень десяти с надстрочными цифрами: $4,708 \cdot 10^{6}$. Приём: `math.floor(math.log10(abs(x)))` — порядок числа (сколько цифр до запятой).

fabry_perot/fmt.py — ticks

```
def ticks(lo, hi, count=6):
    raw = (hi - lo) / count
    if not raw > 0:
        return [lo], 1.0
    mag = 10 ** math.floor(math.log10(raw))
    step = min((k * mag for k in (1, 2, 5, 10)), key=lambda s: abs(math.log(s / raw)))
    first = math.ceil(lo / step - 1e-9) * step
    total = int(math.floor((hi - first) / step + 1e-9)) + 1
    return [first + i * step for i in range(total)], step
```

Деления осей берутся «круглыми»: шаг 1, 2 или 5, умноженный на степень десяти. Сначала находится «сырой» шаг (ширина диапазона / нужное число делений), затем из

кандидатов выбирается ближайший к нему по логарифму: `min(…, key=lambda s: abs(math.log(s / raw)))`.

6. КАК ПРОВЕРЯЕТСЯ ПРАВИЛЬНОСТЬ

В папке `tests` лежат автоматические проверки (библиотека pytest). Они запускаются командой `python -m pytest tests` и при каждой сборке программы на GitHub — на Windows (Python 3.8, Qt 5), macOS и Linux. Если хоть одна проверка не прошла, сборка не выкладывается.

Таблица 3

Что проверяется

Проверка	Смысл
`test_airy_matches_sum_of_beams`	формула Эйри совпадает с прямым сложением 800 лучей до 10^{-9}
`test_measured_curve_matches_theory`	$\Delta\lambda$, w, контраст по графику совпадают с формулами при R от 0,3 до 0,98
`test_ring_radii_are_bright`	на радиусах колец порядок — целое число; $r_2^2 - r_1^2 = f^2 n\lambda \,/\, d$
`test_airy_mean_matches_dense_average`	среднее через первообразную совпадает со средним по миллиону точек
`test_airy_range_contains_every_sample`	все точки внутри столбца лежат между найденными наименьшим и наибольшим
`test_measured_rings_match_formula`	радиусы «по картинке» совпадают с формулой до 10^{-5}
`test_many_thin_rings_are_all_found`	при 22 тысячах колец найдены все
`test_lab.py`	варианты одинаковы при каждом запуске и различны между собой; МНК совпадает с numpy; по идеальным «измерениям» обработка возвращает ту же λ; размытие не сдвигает пики; предел разрешения ≈ w
`test_app.py`	окно открывается, плавный и быстрый переходы, режим лабораторной скрывает λ и не выдаёт её в файлы, журнал записывает точки, лупа, суммарная кривая

Пример — проверка формулы Эйри (файл `tests/test_physics.py`):

```
def test_airy_matches_sum_of_beams():
    R, A = 0.9, 0.01
    delta = np.linspace(0, 4 * np.pi, 2001)
    t = 1 - R - A
    field = t * np.sum([(R * np.exp(1j * delta)) ** k for k in range(800)], axis=0)
    direct = np.abs(field) ** 2
    formula = ph.airy(delta, ph.coefficient_f(R), ph.peak_transmission(R, A))
    assert np.allclose(direct, formula, rtol=1e-9, atol=1e-12)
```

Здесь амплитуда прошедшего света складывается «по определению» — как сумма 800 лучей с множителем $Re^{i\delta}$ (комплексные числа в Python пишутся с `1j`), и результат сравнивается с формулой. $0,9^{800} \approx 10^{-37}$, так что остальные лучи не влияют.

7. ВОПРОСЫ НА ЗАЩИТЕ

Почему результаты «по графику» и «по формуле» немного различаются?

График — это набор точек; пик — самая высокая точка, ширина — по линейной интерполяции. Отличие — сотые доли процента; оно показывает точность численного измерения. Формула (13) для $\Delta\lambda$ сама приближённая ($\lambda^2 \,/\, 2nd$ вместо точной разности $\lambda_{m-1} - \lambda_m$), отсюда отличие в четвёртом знаке.

Зачем нужен numpy?

Он выполняет действия сразу над массивами, в десятки раз быстрее циклов Python. Одна строка `airy(phase(x, d, n), F)` считает пропускание в десятках тысяч точек.

Как программа не теряет узкие пики на графике?

Для каждого столбца пикселей по формуле находится наибольшее и наименьшее значение пропускания в этом столбце; если в столбце есть пик, наибольшее значение равно $T_{\max}$. Поэтому пик виден при любом масштабе.

Откуда берётся яркость пикселя в картине колец?

Это среднее пропускание по пикселю, посчитанное точно через первообразную функции Эйри.

Откуда берутся радиусы колец?

Из точной формулы `ring_radii`: кольцо — это направление, где разность хода равна целому числу длин волн. Картинка колец рисуется той же функцией Эйри, поэтому кольца на ней лежат ровно на этих радиусах.

Что будет, если задать $R + A > 1$?

Пропускание станет нулевым: в `peak_transmission` отрицательное пропускание зеркала заменяется нулём, в окне появится сообщение, что свет не проходит.

Как проверено, что программа считает правильно?

Автоматические тесты сравнивают формулу Эйри с прямым сложением лучей, измерения по графикам — с формулами, быстрые формулы — с расчётом «в лоб» по миллиону точек. Тесты выполняются при каждой сборке на трёх системах.